

Lambda詳細設計書（レンタルスタジオサーチアプリ）

本設計書は、レンタルスタジオサーチアプリにおけるAWS Lambda関数単位の責務・入出力・処理フローを定義する。釣行AIアプリ（fishing-ai-app）の同名資料をベースに、ドメイン変換とスコアリング刷新を反映している。

1. 設計方針

- 1 Lambda = 1責務（単一責任原則）
- API系とバッチ系を分離（SAMの別CodeUriパッケージ、コード共有不可）
- DynamoDBアクセスは原則Lambda経由
- リアルタイム処理は禁止（バッチ中心）
- 実在性に関わる情報（座標・スタジオ名・料金）はGoogle Places APIまたは公式サイトの実テキストのみを情報源とし、AIには生成させない

2. Lambda一覧（API系）

関数名	役割	トリガー	認証
getRecommendationsHandler	おすすめ一覧取得	API Gateway（GET /recommendations）	不要
getStudiosHandler	スタジオ一覧取得	API Gateway（GET /studios）	不要
updateStudioImageHandler	スタジオ写真URLの設定	API Gateway（PUT /studios/{studioid}/image）	必須
getPostsHandler	レビュー取得	API Gateway（GET /posts）	不要

createPostHandler	レビュー投稿の作成（★評価含む）	API Gateway（POST /posts）	必須
updatePostHandler	レビュー投稿の編集（本文・★評価・写真）	API Gateway（PUT /posts/{postId}）	必須
deletePostHandler	レビュー投稿の削除	API Gateway（DELETE /posts/{postId}）	必須
createFavoriteHandler	お気に入り登録	API Gateway（POST /favorites）	必須
deleteFavoriteHandler	お気に入り削除	API Gateway（DELETE /favorites/{studioId}）	必須
getFavoritesHandler	お気に入り取得	API Gateway（GET /favorites）	必須
presignUploadHandler	S3アップロード用の署名付きURL発行	API Gateway（POST /uploads/presign）	必須
postChatHandler	AIチャットへのメッセージ送信・Claude応答生成・メッセージ編集	API Gateway（POST /chat）	必須
getChatsHandler	チャット履歴一覧取得	API Gateway（GET /chats）	必須
getChatHandler	特定チャットの全メッセージ取得	API Gateway（GET /chats/{chatId}）	必須
deleteChatHandler	チャットの削除	API Gateway（DELETE /chats/{chatId}）	必須
getMyProfileHandler	自分の表示名（プロフィール	API Gateway（GET /me）	必須

	ル) 取得		
putMyProfileHandler	表示名の設定・更新	API Gateway (PUT /me)	必須

3. Lambda一覧（バッチ系）

関数名	役割	トリガー
generateStudioScoreBatch	新規スタジオ探索 + スコア生成 + AI推薦理由生成	EventBridge（毎日 AM6:00 JST）／ adminRunAiBatch経由
adminRunAiBatch	フロントの「AI分析を実行」ボタンから generateStudioScoreBatchを起動する薄いラッパー。1日3回のレート制限	API Gateway (POST /admin/run-ai-batch、認証必須)
discoverStudiosBatch	Google Places APIで新規レンタルスタジオ候補を探索しStudiosテーブルへ追加	adminRunStudioDiscovery 経由のみ（自動スケジュールなし）
adminRunStudioDiscovery	フロントの「現在地から新スタジオを探す」ボタンからdiscoverStudiosBatchを起動する薄いラッパー （adminRunAiBatchと同一コードを BATCH_FUNCTION_NAME 環境変数の差し替えで再利用）。1日10回のレート制限	API Gateway (POST /admin/run-studio-discovery、認証必須)

釣行AIアプリのtackle_shops.py (searchTackleShops) に相当する機能は無い（v1では対象外）。

4. 個別設計

getRecommendationsHandler

目的：ユーザーにおすすめスタジオ一覧（スコア降順）を返却。フロント側でTOP3とその他に分けて表示する。

処理フロー

1. RecommendationsTable全件取得
2. StudiosTable全件取得し studioId で結合（studio フィールドとして付与）
3. score降順でソート
4. レスポンス返却

出力（1件分の例）

```
{
  "studioId": "studio-6ae9be37",
  "score": 78.4,
  "facilityTags": ["鏡張り", "フローリング", "音響設備"],
  "capacityCategory": "小グループ向け（6～10人）",
  "reason": "ロコミ評価が高く、駅から近いのが魅力です。",
  "distance": 12.3,
  "stationDistanceKm": 0.49,
  "cost": 1000,
  "ratingScore": 88.0,
  "popularityScore": 45.0,
  "updatedAt": "2026-07-29T02:10:00+00:00",
  "studio": { "studioId": "studio-6ae9be37", "name": "...", "lat": 35.6, "lng": 139.7, ... }
}
```

generateStudioScoreBatch

目的：全スタジオのスコア計算+AI推薦理由生成。handlerの冒頭で discover_studios.run_discovery() を呼び出し、新規スタジオ探索も内包する（「AI分析を実行」が新スタジオ探索とスコア再計算の両方を兼ねる）。

スコアロジック（現行）

```
score = rating_score      * 0.4  # Places rating(0-5) を 0-100 に正規化。無ければ中立の
      + popularity_score  * 0.2  # user_ratings_total を0-100に正規化（200件で頭打ち）
      + appeal_score       * 0.2  # 設備タグの一致数によるヒューリスティック
```

```
- station_dist_norm    * 0.1    # 最寄り駅からの距離 (STATION_DISTANCE_NORM_KM=1.5kmで  
- cost_norm            * 0.1    # 利用料金 (5000円で頭打ち。不明なら0扱い)
```

重み係数の合計は0.8のため、満点条件でもスコア上限は80点（釣行AIアプリの `calc_score()` と同じ構造）。

距離を `station_distance_km` に変更した経緯: 当初は釣行AIアプリと同じく「基準地点（東京駅）からの距離」を使っていたが、全国のスタジオを対象にする本アプリではこの指標に意味が薄かった。また、レンタルスタジオは天気・潮汐のような「毎日変わる」要素が無いため、スコアの中で実質的に唯一動きうる「距離」要素を、ユーザーの現在地に依存しないスタジオ自体の固有属性（最寄り駅からのアクセス、Places Nearby Searchで取得）に置き換えた。基準地点からの距離（`distanceKm`）自体は削除せず、フロントの「現在地から探す」機能（`recalcScoreForDistance`）が引き続き使用するため `distance` として Recommendationsへ保存し続けている。この2つの「距離」は別概念であり混同しないよう注意。

reason使い回しによるAPI呼び出し削減

スタジオ数が数十～百件規模になると、毎回全件についてAIに推薦理由(`reason`)を書かせるのはコスト・時間の面で無駄が大きい。`_should_regenerate_reason(existing, new_score)`により、以下のいずれにも該当しないスタジオは前回の`reason`文をそのまま使い回し、Claude呼び出しをしない:

- そのスタジオの推薦データが存在しない（初回）
- スコアが前回から`REASON_SCORE_CHANGE_THRESHOLD`（10.0）以上動いた
- 前回の生成から`REASON_MAX_AGE_DAYS`（3日）以上経過している

スコア自体（DynamoDBの`updatedAt`含む）は使い回し対象外で毎回必ず再計算・更新するため、フロント（`useRecommendations.ts`）の「全スタジオの`updatedAt`が実行開始時刻より新しくなったら完了」というポーリング判定には影響しない。この仕組みは釣行AIアプリでGemini無料枠の枯渇障害をきっかけに導入されたものを、v1から最初に組み込んでいる。

`discover_studios.run_discovery()`

目的：新規レンタルスタジオ候補をGoogle Places APIで探索し、Studiosテーブルに追加する中核ロジック。 `generateStudioScoreBatch` のhandler冒頭（位置指定なし）

と、discoverStudiosBatch のhandler（「現在地から新スタジオを探す」ボタン経由、位置指定あり）の両方から呼ばれる共通関数。

処理

Studios全件取得（重複判定用）

→ Google Places API Text Search

- ・位置指定なし：“ダンススタジオ レンタル”等の全国向け複数クエリ
- ・位置指定あり：“レンタルスタジオ”1クエリ + location/radius(15km)パラメータで近傍に絞

→ 既存スタジオと緯度経度がhaversine距離300m以内なら重複として除外

→ 新規候補ごとに（既存スタジオはここに到達しないため一回きりのコスト）：

- ・Claude APIで店名・住所から想定される設備を推測（guess_facilities）
- ・Claude APIで店名・住所から収容人数の目安を推測（guess_capacity、3区分）
- ・Places Nearby Search（type=train_station, rankby=distance）で最寄り駅までの距離を算出
- ・Place Details（fields=website）で公式サイトURLを取得できればサイトを取得し、
Claudeに実際に書かれている料金（円/時間）の抽出を試みる（scrape_price_from_website）

→ 基準地点（位置指定時はその座標、なければ東京駅）からのdistanceKmを算出（現在地から探す）

→ Studiosテーブルへput_item（placeIdも保存し、将来の再取得に備える）

座標・スタジオ名などの実在性に関わる情報はGoogle Places APIの結果のみを採用し、LLM（Claude）には生成させない（存在しないスタジオをAIが作り出すハルシネーションを避けるため）。Claudeは設備・収容人数の推測、公式サイトに実在するテキストからの料金抽出という、「間違っても実害が小さい／根拠となる実データが存在する」情報生成にのみ用いる。料金抽出プロンプトは「サイトに明記されていないければ"不明"とだけ答える」よう明示的に指示しており、金額をAIが推測で作り出すことはない設計にしている。

generate_studio_score.py が discover_studios.py の run_discovery() を直接importして呼び出すため、逆方向のimport（discover_studios→generate_studio_score）は循環importになり不可能。共有ヘルパー（DynamoDB/SSMアクセス・HTTP GET）は batch_common.py に切り出し、両モジュールがそこから import する。

backfill_studio_details.py（一回限りのメンテナンススクリプト）

目的：収容人数・最寄り駅距離・料金のスコアリング項目を追加する前に発見済みだった既存スタジオへ、これらのフィールドを後追いで埋める。run_discovery()は300m以内の重複を検出済み候補としてスキップするため、既存スタジオはこの拡張の対象に自然には到達しない。

処理

StudiosTable全件scan → capacityCategory/nearestStationDistanceMが未設定の項目を抽出
→ 各項目について:
 guess_capacity() / find_nearest_station_distance_m() を実行
 placeIdが無ければ Places Find Place From Text (studio名+座標) で復旧を試みる
 placeIdが取得できればfetch_place_website()→scrape_price_from_website()で料金を試みる
→ DynamoDB update_item で該当フィールドのみ更新 (他フィールドは変更しない)

本番のDynamoDBテーブル・実際のGoogle Places/Anthropic APIキーを直接使う (motoでのモックは行わない) ため、Lambdaとしてはデプロイせず、ローカル端末のAWS認証情報でpython backfill_studio_details.py として手動実行する。--dry-runオプションでDynamoDBを更新せず結果のみ確認できる。

presignUploadHandler / updateStudioImageHandler / createPostHandler

目的: スタジオ写真・レビュー写真のアップロードと、レビュー投稿の作成。

処理フロー (写真アップロード)

1. フロントが POST /uploads/presign を呼び、S3への署名付きPOSTフォーム (uploadUrl + upload)
2. フロントがそのフォームへ画像ファイルをmultipart/form-dataでPOST (Lambda/API Gatewayを経
3. アップロード完了後、公開URL (publicUrl) を
 PUT /studios/{studioId}/image (スタジオ写真) または
 POST /posts のimageUrl (レビュー写真) または
 POST /chat のimageUrl (チャット添付) として保存

アップロード先S3バケット (UploadsBucket) はGetObjectのみ公開許可、書き込みは署名付きURL経由のみに 限定している。generate_presigned_postのConditionsに ["content-length-range", 1, 8388608] (8MB) を指定し、S3側にアップロード上限を強制させている。

postChatHandler / getChatsHandler / getChatHandler

目的: 写真添付にも対応したAIチャット機能。写真添付 (端末アルバムから選択 or その場で撮影、どちらも同じ<input type="file">で実現し、撮影用は capture="environment"属性を付与) を チャット入力の一部として統合している。

処理フロー（POST /chat）

1. UsageTableでレート制限チェック（1日30件、DAILY_CHAT_LIMIT）
2. editIndexが指定されていれば、該当インデックス以降のmessagesを切り捨ててから処理を続行（
3. chatId が未指定なら新規チャットとしてuuid発行、指定されていればChatsTableから既存message
4. ユーザーメッセージ（+ imageUrlがあればS3から画像バイトを取得）をmessagesに追記
5. _build_studios_context(lat, lng) がStudios・Recommendationsをscan→結合→ソート→上位N件をテキスト化し、実データグラウンディング用のコンテキストとして構築
6. Claudeへ直近の会話履歴 + 今回のメッセージ（+画像）+ 実データコンテキストをマルチモーダル（システムプロンプトで「ダンス・ヨガの専門家アシスタント」という役割を明示。「データに無いスタジオは作り出さない」という指示も明記）
7. 応答をmessagesに追記し、ChatsTableへput_item（アイテム全体を上書き）
8. { chatId, reply, updatedAt } を同期的に返却

他のAI機能（generateStudioScoreBatch等）はAPI Gateway の29秒タイムアウトを避けるため 非同期トリガー+ポーリング方式を採っているが、チャットはUX上ポーリングが合わないため **同期呼び出しのまま**とし、Lambda Timeoutを29秒より短い25秒に設定することで対処している（テキスト+画像1枚のClaude呼び出しは通常数秒～十数秒で収まる想定）。

GET /chats は履歴パネルの一覧表示用に messages を含まない軽量のレスポンス（chatId/title/updatedAtのみ）を返す。GET /chats/{chatId} は履歴から会話を再開する際に全メッセージを返す。「新しい会話」開始時はDBに何も書き込まず、次にメッセージを送信した時点で初めて新規チャットとして保存される。

メッセージ編集: 自分の発言に表示される鉛筆アイコンから本文を訂正して再送信できる。フロント（ChatPage.tsx）が編集対象のインデックスを保持し、送信時にeditIndexとしてバックエンドへ渡す。postChatHandlerは該当インデックス以降の全メッセージ（訂正対象の発言+それ以降のAI応答）を切り捨ててから、訂正後の本文で通常の送信フローを再実行する（AIの回答も作り直される）。

コスト保護レート制限（admin_trigger.py）

目的：AI分析実行・現在地から新スタジオ探索はいずれもPlaces/Claude APIの呼び出しを伴い課金対象のため、AI相談（postChatHandlerのDAILY_CHAT_LIMIT）と同じ思想で1ユーザー1日あたりの上限を設けている。api/とbatch/はSAMの別デプロイパッケージのためhandlers.pyのロジックは共有できず、batch_common.pyにget_user_id・check_and_increment_daily_usageとして複製している。

関数	レート制限アクション名	1日の上限	設定方法
adminRunAiBatch	ai-batch	3回	template.yamlの RATE_LIMIT_ACTION/RATE_LIMIT_DAILY 環境変数
adminRunStudioDiscovery	studio-discovery	10回	同上（同じadmin_trigger.pyを異なる環境変数で再利用）

上限に達している場合はバッチ起動・Places API検索を一切行わずに429（{"status": "rate_limited", "message": "..."}）を返す。カウンタはUsageTable（userId + dateKey）にADD式でアトミック加算し、TTLで翌々日ごろ自動的に古いカウンタが削除される（postChatHandlerと全く同じ仕組み）。

getMyProfileHandler / putMyProfileHandler

目的：ユーザーが自分の表示名を設定・編集できるようにする。UsersTable（userId 単一PK）に displayName・email・createdAt・updatedAt を保存する。GET /me は未登録でも404にせず displayName: null で200を返す。

投稿一覧（getPostsHandler）は、SpotsTable/FavoritesTableの結合と同じ「NoSQLはJOINできないので アプリ側でuserId→表示名の辞書を作って結合する」パターンで UsersTableをscanし、各投稿に authorName（未設定ユーザーは"匿名"）を付与する。

5. 非機能要件

- API応答：1～2秒以内（AI分析・チャットを除く）
- バッチ：日次1回 + 手動実行
- エラーハンドリング：外部API失敗時はフォールバック値で処理継続、例外を投げずバッチ全体を止めない

6. セキュリティ

6.1 認証範囲

閲覧は未ログインでも可能（おすすめ・地図・スタジオ一覧・投稿一覧）。操作（お気に入り・投稿作成/削除・AI相談・AI分析実行・新スタジオ探索）はログイン必須という方針とした（誰でも見られるアプリという性質を保ちつつ、書き込み系・課金系だけを保護する狙い。釣行AIアプリの方針をそのまま踏襲）。

6.2 認証フロー

フロント (react-oidc-context, Authorization Code + PKCE)
→ Cognito Hosted UI (/oauth2/authorize) へリダイレクト
→ Hosted UI上でGoogleログイン (Cognito側のGoogle IdP連携経由)
→ Cognitoの/oauth2/idpresponseへコールバック → 認可コードをid_tokenに交換
→ 以後のAPIリクエストに Authorization: Bearer <id_token> を付与
→ API Gateway の CognitoAuthorizer (User Pool Authorizer) がid_tokenを検証
→ 検証成功時、Lambda側イベントに event.requestContext.authorizer.claims.sub が入る
→ handlers.py の _get_user_id(event) がこれを実ユーザーIDとして使用

CognitoはOIDC標準の end_session_endpoint をディスカバリドキュメントに含めないため、ログアウトは react-oidc-context の標準 signoutRedirect() ではなく、Hosted UI独自の /logout エンドポイントへ直接遷移する方式で実装している

(frontend/src/auth/authConfig.ts の cognitoSignOut()。釣行AIアプリと同じ実装)。

User PoolはCognitoドメインstudio-search-app-<AWSアカウントID>.auth.ap-northeast-1.amazonaws.comを使用 (アカウントID込みの標準形式)。釣行AIアプリで短いカスタムプレフィックスへの変更を試みてデプロイ失敗 (Cognitoは1つのUser Poolに同時に2つのドメインを持ってない) した教訓があるため、本アプリでは当初からこの標準形式を採用し、変更は行っていない。

6.3 所有者チェック

Favorites/Chatsテーブルは userId がpartition keyの一部のため、他人のuserIdではそもそも該当データにアクセスできない設計 (追加の所有者チェック不要)。一方 Postsテーブルは postId 単一PKで所有者情報がキーに含まれないため、deletePostHandler/updatePostHandlerで更新・削除前に get_item して投稿の userId とリクエスト元の userId を比較し、一致しなければ403を返す明示的なチェックを行っている。

7. テスト戦略

- バックエンド: pytest。純粋関数（`calc_score`・`haversine_km`・`guess_capacity`・`normalize_*`等）の単体テストに加え、`moto`でDynamoDB/S3をモックしたハンドラーレベルのテスト（投稿・チャットの作成→削除ライフサイクル、写真アップロード等）。67件からスコアリング刷新で7件追加し67件
- フロントエンド ユニットテスト: Vitest + React Testing Library（`utils/`の純粋関数、`RecommendationCard`のレンダリングテスト）。18件
- フロントエンド E2E・VRT: 釣行AIアプリでは導入していたPlaywrightによるE2E/VRTを、v1のスコープでは見送っている（`package.json`に`@playwright/test`を含めていない）
- CI: `.github/workflows/test.yml`が `pull_request` → `main` をトリガーに実行される（`deploy.yml`の `push` → `main` トリガーとは分離。デプロイは行わずテストのみ。フロントエンドジョブは`npm run build`も実行し型エラーを検出する）

8. まとめ

Lambdaは「API処理」と「バッチ処理」に完全分離し、スケーラブルかつ低コストな構成とする。釣行AIアプリのアーキテクチャを流用しつつ、ドメイン固有の判断（最寄り駅距離・収容人数・公式サイトからの料金抽出）を新たに組み込んでいる。